COMPILER DESIGN

Theory Answers & Lex/Flex Practical Programs

Complete Answer Booklet – Sets 1 to 4

Includes simple explanations, short supporting definitions, labelled text diagrams, algorithms, Lex programs, and sample execution notes.

Student-friendly • Exam-ready • Easy-to-revise format

How to use this booklet

The theory answers are written in straightforward language with a few supporting definitions so they are long enough for examination writing without becoming unnecessarily complicated. Practical programs use standard Lex/Flex conventions. Save each program with a **.l** extension and run it using the commands shown below.

General commands (Linux / WSL):

```
flex program.l
gcc lex.yy.c -o program -lfl
./program input.txt
```

For programs that read from a file, pass the filename as a command-line argument. The programs below use `yyin` for this purpose. If your lab uses classic Lex, the same basic rules usually work; the Flex-specific option `%option noyywrap` avoids needing a separate `yywrap()` function.

Note: Lex rules are matched using patterns. When two rules match the same amount of text, the rule written first is selected. This is why keyword rules are placed before the general identifier rule.

SET 1

Q1. Theory – Compiler vs. Interpreter

Compiler: A compiler is system software that translates a complete program written in a high-level language into machine code or another target form before the program is executed. It reports errors during the translation process and may produce an executable file.

Interpreter: An interpreter is system software that reads and executes a program statement by statement, usually without producing a separate permanent machine-code executable.

Working process

```
COMPILER:
Source Program → Compiler → Object / Machine Code → Execution → Output

INTERPRETER:
Source Program → Interpreter → Execute statement by statement → Output
```

Differences

Basis	Compiler	Interpreter
Translation	Translates the program as a whole (or large units).	Translates and executes statement by statement.
Output	Usually creates object code or an executable.	Usually does not create a separate executable.
Speed	Compiled program generally runs faster.	Execution is generally slower due to runtime translation.
Error reporting	Reports errors during compilation; often lists several errors.	Often stops when it reaches an error, depending on the interpreter.
Examples	C and C++ are commonly compiled.	Traditional Python and JavaScript implementations use interpretation/J

Examples

In C, a compiler such as GCC translates `hello.c` into an executable before it runs. In a traditional Python interpreter, a command such as `print("Hello")` is read and executed by the Python runtime. Modern language implementations may combine compilation and interpretation, so the distinction is a simplified one.

Conclusion: A compiler translates before execution, while an interpreter performs translation as part of execution. Both convert human-readable instructions into actions the computer can perform.

Q2. Theory – Phases of a Compiler

A compiler is a system program that converts source code into target code. It performs this task in a number of phases. Each phase takes input from the previous phase, processes it, and passes the result to the next phase.

Neat labelled diagram

```
          SOURCE PROGRAM
                 |
                 v
        1. LEXICAL ANALYSIS
           (tokens)
                 |
                 v
        2. SYNTAX ANALYSIS
           (syntax tree)
                 |
                 v
       3. SEMANTIC ANALYSIS
        (meaning / checks)
                 |
                 v
    4. INTERMEDIATE CODE GENERATION
        (intermediate code)
                 |
                 v
        5. CODE OPTIMIZATION
         (improved code)
                 |
```

```
            v
        6. CODE GENERATION
          (target code)
                |
                v
           TARGET PROGRAM

    SYMBOL TABLE and ERROR HANDLER
    support the compiler phases.
```

Explanation of phases

• **1. Lexical analysis:** The source program is read character by character. The characters are grouped into tokens such as keywords, identifiers, operators, and constants. Example: `sum = a + 5` gives tokens `sum`, =, a, +, and 5.

• **2. Syntax analysis:** The parser checks whether the token sequence follows the grammar of the language. It builds a parse tree or syntax tree and reports syntax errors such as a missing semicolon.

• **3. Semantic analysis:** This phase checks the meaning of the program. It checks declarations, data types, scope, and compatible operations. For example, it can detect an invalid operation between incompatible data types.

• **4. Intermediate code generation:** The compiler converts the program into a machine-independent intermediate representation. Example: `x = a + b * c` may become `t1 = b * c; t2 = a + t1; x = t2`.

• **5. Code optimization:** It improves intermediate code to make it faster or smaller without changing the program's result. It may remove unnecessary calculations or repeated expressions.

• **6. Code generation:** The optimized intermediate code is converted into target assembly or machine code. Register allocation and instruction selection may be performed here.

Supporting components: The symbol table stores information about identifiers, such as their names, types, and scope. The error handler detects and reports errors found in different phases.

Conclusion: These phases work together to convert source code into an efficient target program while checking its correctness.

Q3. Practical – Experiment 3: Recognize identifiers, keywords and operators

Aim: To write a Lex program that identifies keywords, identifiers, and operators from the input source code.

Definition: A keyword is a reserved word with a fixed meaning in a programming language, such as `int` or `while`. An identifier is a user-defined name, such as a variable or function name. An operator is a symbol that performs an operation, such as +, =, or *.

Program

```
%{
#include <stdio.h>
%}

%%
"int"|"float"|"char"|"if"|"else"|"while"|"for"|"return"|"void"
                { printf("Keyword: %s\n", yytext); }

[a-zA-Z_][a-zA-Z0-9_]*
                { printf("Identifier: %s\n", yytext); }

"=="|"!="|"<="|">="|"++"|"--"|"&&"|"||"|"+"|"-"|"*"|"/"|"="|"<"|">"
                { printf("Operator: %s\n", yytext); }

[ \t\n]+        ;
.               { printf("Other: %s\n", yytext); }
%%

int main(void)
{
    yylex();
    return 0;
}
```

Explanation: The first rule recognizes reserved keywords. The second rule recognizes identifiers beginning with a letter or underscore. The third rule recognizes common arithmetic, relational, logical, and assignment operators.

Whitespace is ignored.

Sample input: `int total = a + 10;` **Expected output:** Keyword: int; Identifier: total; Operator: =; Identifier: a; Operator: +.

SET 2

Q1. Theory – Bootstrapping and Self-Hosting

1. What is bootstrapping?

Bootstrapping is the process of developing a compiler for a programming language with the help of an existing compiler or a simpler tool. In simple words, it is a method of building a new compiler step by step, using an available compiler as support.

2. What is self-hosting?

A self-hosting compiler is a compiler that is written in the same programming language that it compiles. For example, a C compiler written in C is called a self-hosting C compiler.

3. Significance of bootstrapping in compiler design

• It helps developers create a compiler for a new language using an existing language or compiler.

• It makes compiler development easier by allowing the compiler to be improved in stages.

• It helps test the language because the compiler itself is written using that language.

• It supports portability: once a compiler is available on one machine, it can help build a compiler for another machine.

• It supports self-hosting, where a compiler can compile its own source code.

Simple bootstrapping process

```
Existing compiler / small compiler
              |
              v
   Compile the first version of
       the new compiler
              |
              v
    Use the new compiler to
    compile its improved version
              |
              v
      Self-hosting compiler
```

Conclusion: Bootstrapping is the development technique, while self-hosting describes a compiler that is written in the language it compiles. Bootstrapping is an important way to reach a self-hosting compiler.

Q2. Practical – Experiment 6: Count numbers and keywords in a file

Aim: To count the total number of numeric constants and keywords present in a given source file.

Definition: A number is a numeric value such as `25` or `3.14`. A keyword is a reserved word of a programming language, such as `int`, `if`, or `return`.

Program

```
%{
#include <stdio.h>
int numbers = 0, keywords = 0;
%}

%%
"int"|"float"|"char"|"double"|"if"|"else"|"while"|"for"|"return"|"void"
                { keywords++; }

[0-9]+(\.[0-9]+)?  { numbers++; }

[a-zA-Z_][a-zA-Z0-9_]* ;
[ \t\n]+          ;
.                 ;
%%

int main(int argc, char *argv[])
```

```
{
    if (argc > 1) {
        yyin = fopen(argv[1], "r");
        if (!yyin) {
            perror("File open error");
            return 1;
        }
    }
    yylex();
    printf("Total numbers = %d\n", numbers);
    printf("Total keywords = %d\n", keywords);
    if (yyin != stdin) fclose(yyin);
    return 0;
}
```

Explanation: The keyword rule increases the keyword counter. The numeric rule accepts integers and simple decimal numbers and increases the number counter. Other identifiers and symbols are ignored.

Run: `flex count.l`, then `gcc lex.yy.c -o count -lfl`, then `./count input.txt`.

Q3. Practical – Experiment 8: Count occurrences of each character in a file

Aim: To count how many times each character occurs in a given text file.

Definition: A character is a single unit of text, such as a letter, digit, space, or punctuation mark. Character frequency is the number of times a particular character appears in the input.

Program

```
%{
#include <stdio.h>
int freq[256] = {0};
%}

%%
.       { freq[(unsigned char)yytext[0]]++; }
\n      { freq[(unsigned char)yytext[0]]++; }
%%

int main(int argc, char *argv[])
{
    int i;

    if (argc > 1) {
        yyin = fopen(argv[1], "r");
        if (!yyin) {
            perror("File open error");
            return 1;
        }
    }

    yylex();

    printf("Character frequencies:\n");
    for (i = 0; i < 256; i++) {
        if (freq[i] > 0) {
            if (i == '\n')
                printf("\\n : %d\n", freq[i]);
            else if (i == '\t')
                printf("\\t : %d\n", freq[i]);
            else if (i == ' ')
                printf("space : %d\n", freq[i]);
            else
                printf("%c : %d\n", i, freq[i]);
        }
    }

    if (yyin != stdin) fclose(yyin);
    return 0;
}
```

Explanation: The dot rule counts ordinary characters and the newline rule counts line breaks. Each character is converted to an unsigned array index and its frequency is increased. The final loop prints only characters that occurred at least once.

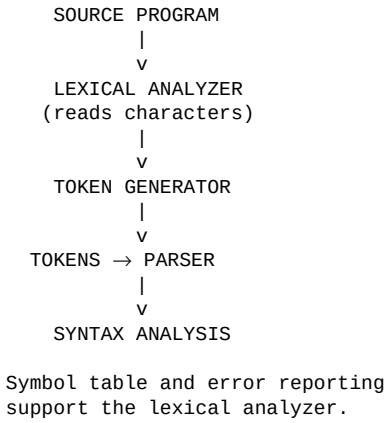
Important: In Flex, the dot does not match a newline, so a separate newline rule is necessary.

SET 3

Q1. Theory – Lexical Analysis

Lexical analysis is the first phase of a compiler. It reads the source program character by character and groups the characters into meaningful units called tokens. The program that performs lexical analysis is called a lexical analyzer or scanner.

Neat labelled diagram



Functions of lexical analysis

• Reads the source program as a stream of characters.

• Groups characters into tokens such as keywords, identifiers, constants, and operators.

• Ignores unnecessary whitespace and comments in many languages.

• Enters or looks up identifier information in the symbol table.

• Reports lexical errors, such as an invalid character or malformed token.

1. Token

A token is a category or class of lexical units recognized by the lexical analyzer. Examples include KEYWORD, IDENTIFIER, NUMBER, and OPERATOR. For example, `count` may be classified as an IDENTIFIER token.

2. Lexeme

A lexeme is the actual sequence of characters in the source program that matches a token pattern. In `int count = 10;`, the lexemes are `int`, `count`, `=`, `10`, and `;`.

3. Pattern

A pattern is a rule that describes the form of lexemes belonging to a token. For example, the pattern `[A-Za-z][A-Za-z0-9]*` describes an identifier that starts with a letter and may be followed by letters or digits.

Term	Meaning	Example
Token	Category of a lexical unit	IDENTIFIER
Lexeme	Actual text in the program	total
Pattern	Rule describing valid lexemes	[A-Za-z][A-Za-z0-9]*

Example: In the statement `int age = 20;`, `age` is a lexeme, IDENTIFIER is its token, and the identifier pattern describes how names such as `age` are formed.

Q2. Practical – Experiment 7: Count operators in a file

Aim: To write a Lex program that counts the operators present in a source file.

Definition: An operator is a symbol that performs an operation on one or more operands. Examples include arithmetic operators (+, -, *, /), relational operators (==, <, >), and logical operators (&&, ||).

Program

```
%{
#include <stdio.h>
int count = 0;
%}

%%
"=="|"!="|"<="|">="|"++"|"--"|"&&"|"||"|"+"|"-"|"*"|"/"|"%"|"="|"<"|">"|"!"
                { count++; }

[ \t\n]+        ;
.               ;
%%

int main(int argc, char *argv[])
{
    if (argc > 1) {
        yyin = fopen(argv[1], "r");
        if (!yyin) {
            perror("File open error");
            return 1;
        }
    }
    yylex();
    printf("Total operators = %d\n", count);
    if (yyin != stdin) fclose(yyin);
    return 0;
}
```

Explanation: The first rule matches common one-character and multi-character operators. Every match increases the counter by one. Longer operators are listed before shorter ones for clarity; Lex's longest-match rule also ensures that == is matched as one operator rather than two separate equals signs.

Q3. Practical – Experiment 5: Check whether a string is a keyword

Aim: To check whether the input string is a reserved keyword of the language.

Definition: A keyword is a reserved word that has a predefined meaning in a programming language. It cannot normally be used as a user-defined identifier. Examples are `int`, `if`, and `while`.

Program

```
%{
#include <stdio.h>
int found = 0;
%}

%%
"int"|"float"|"char"|"double"|"if"|"else"|"while"|"for"|"return"|"void"
                { found = 1; }

[a-zA-Z_][a-zA-Z0-9_]* ;
[ \t\n]+        ;
.               ;
%%

int main(void)
{
    printf("Enter a string: ");
    yylex();

    if (found)
        printf("The string is a keyword.\n");
    else
        printf("The string is not a keyword.\n");

    return 0;
}
```

Explanation: The keyword rule sets the flag to 1 when a reserved word is found. The identifier rule accepts an ordinary word without changing the flag. This simple program is intended for one input word; enter only one string

when testing.

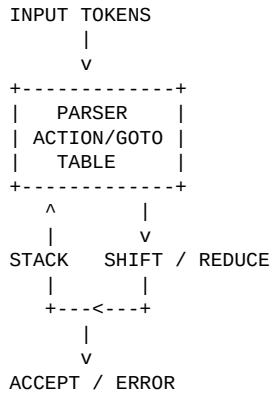
Example: Input: `while` → Output: The string is a keyword.

SET 4

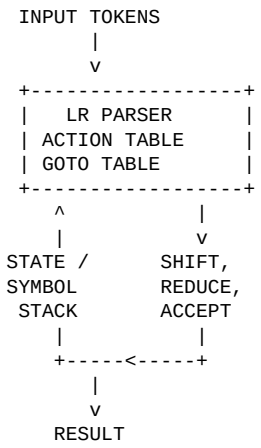
Q1. Theory – SLR and LR Parsers

A parser is a part of the compiler that checks whether the sequence of tokens follows the grammar of a programming language. SLR and LR are bottom-up parsing techniques. They scan input from left to right and construct a rightmost derivation in reverse.

SLR parser diagram



LR parser diagram (general LR framework)



Differences between SLR and LR

Basis	SLR Parser	Canonical LR(1) Parser
Full form	Simple LR	Canonical LR(1)
Items	Uses LR(0) items.	Uses LR(1) items with one-symbol lookahead.
Lookahead	Uses FOLLOW sets to decide reductions.	Uses item-specific lookahead symbols.
Parsing power	Less powerful; may have conflicts for some grammars.	More powerful; handles a larger class of grammars.
Table size	Usually smaller and simpler.	Usually larger because it has more states.
Complexity	Easier to construct by hand.	More complex to construct.

Working: Both parsers use a stack and parsing tables. The ACTION table tells the parser whether to shift an input symbol, reduce using a grammar production, accept the input, or report an error. The GOTO table gives the next state after a non-terminal is recognized. SLR uses FOLLOW sets for reductions, while canonical LR(1) stores more precise lookahead information in its items.

Note: The term LR is sometimes used broadly for the LR family. In this comparison, LR means canonical LR(1), which is the common comparison with SLR in compiler-design courses.

Q2. Theory – FLEX and BISON

FLEX: FLEX is a fast lexical-analyzer generator. It takes token patterns written as regular expressions and actions written in C, then generates a scanner program. It is commonly used to recognize keywords, identifiers, numbers, and operators.

BISON: BISON is a parser generator. It takes grammar rules and associated actions and generates a parser, usually for syntax analysis. It is commonly used with a scanner generated by Flex.

Working of FLEX and BISON

```
Source Code
    |
    v
 FLEX scanner  <--- regular-expression rules
    |
    | token stream
    v
 BISON parser  <--- grammar / production rules
    |
    v
 Syntax checking / parse result
```

Differences

Basis	FLEX	BISON
Purpose	Lexical analysis (scanner).	Syntax analysis (parser).
Input specification	Regular expressions and actions.	Context-free grammar rules and actions.
Output	A lexical analyzer, commonly lex.yy.c.	A parser, commonly parser.tab.c and parser.tab.h.
Recognizes	Tokens such as identifiers, numbers, and operators.	Grammar structures such as expressions and statements.
Typical role	Finds the words/tokens in source code.	Checks how tokens are arranged.

Combined use: FLEX reads source characters and returns tokens to BISON. BISON receives those tokens and checks whether they follow the grammar. Therefore, FLEX handles the words of the language, while BISON handles their grammatical structure.

Q3. Theory – Context-Free Grammar (CFG)

A Context-Free Grammar (CFG) is a formal system used to describe the syntax or structure of a language. It consists of a set of production rules that describe how strings in the language can be generated. CFGs are widely used in compiler design to define programming-language syntax.

Components of CFG

A CFG is commonly written as G = (V, T, P, S), where:

• **V – Variables / non-terminals:** Symbols that can be expanded, such as S, A, or Expr.

• **T – Terminals:** Actual symbols of the language that cannot be expanded, such as a, b, +, or id.

• **P – Production rules:** Rules that describe how a non-terminal can be replaced by a sequence of terminals and/or non-terminals.

• **S – Start symbol:** The non-terminal from which derivation begins.

Production rules and derivation

A production rule has the form $A \rightarrow \alpha$, where A is a single non-terminal and α is a sequence of terminals and/or non-terminals. A derivation applies production rules repeatedly, starting from the start symbol, until a string of terminals is produced.

Example CFG

```
G = (V, T, P, S)

V = {S}
```

```
T = {a, b}
P = {S -> aSb, S -> ab}
Start symbol = S
```

Derivation of the string `aabb`:

```
S
=> aSb       (using S -> aSb)
=> aabb      (using S -> ab)
```

This grammar generates strings containing one or more a's followed by the same number of b's, such as `ab`, `aabb`, and `aaabbb`.

Conclusion: CFGs provide a clear mathematical way to describe programming-language syntax and are used by parsers during syntax analysis.

Q4. Practical – Experiment 2: Count words, spaces and lines in a text file

Aim: To write a Lex program that counts the total number of words, spaces, and lines in a given text file.

Definition: A word is a sequence of non-whitespace characters. A space is a blank-space character. A line is a row of text ending with a newline character. In this program, tabs are not counted as spaces, and words are separated by whitespace.

Program

```
%{
#include <stdio.h>
int words = 0, spaces = 0, lines = 0;
%}

%%
[^ \t\n]+    { words++; }
" "          { spaces++; }
\n           { lines++; }
\t           ;
%%

int main(int argc, char *argv[])
{
    if (argc > 1) {
        yyin = fopen(argv[1], "r");
        if (!yyin) {
            perror("File open error");
            return 1;
        }
    }

    yylex();

    printf("Total words = %d\n", words);
    printf("Total spaces = %d\n", spaces);
    printf("Total lines = %d\n", lines);

    if (yyin != stdin) fclose(yyin);
    return 0;
}
```

Explanation: The first rule counts each continuous group of non-space, non-tab, and non-newline characters as one word. The second rule counts each ordinary blank space. The third rule counts newline characters as lines. Tabs are ignored and are not included in the space count.

Note about lines: This program counts newline characters. Therefore, a final line without a newline at the end of the file is not counted as an additional line. This definition is simple and suitable for a basic Lex practical.

Run: Save as `words.l`, compile with Flex and GCC, then run `./words input.txt`.

QUICK REVISION

Topic	Remember
Compiler	Translates source code before execution; commonly creates target code.
Interpreter	Reads and executes instructions during runtime.
Compiler phases	Lexical → Syntax → Semantic → Intermediate Code → Optimization → Code Generation.
Bootstrapping	Building a compiler using an existing compiler/tool, step by step.
Self-hosting	A compiler written in the language that it compiles.
Token	Category of a lexical unit.
Lexeme	Actual character sequence in the source.
Pattern	Rule that describes valid lexemes.
SLR	LR(0) items; FOLLOW sets guide reductions.
Canonical LR(1)	LR(1) items; item-specific lookahead.
FLEX	Generates lexical analyzers from regular-expression rules.
BISON	Generates parsers from grammar rules.
CFG	G = (V, T, P, S): variables, terminals, productions, start symbol.

Practical exam checklist

• Write the aim and a one- or two-line definition before the code.

• Keep keyword rules before the general identifier rule.

• Remember that Flex's dot (.) does not match newline; add a separate newline rule when required.

• For file-based programs, open the file using `fopen()` and assign it to `yyin`.

• Compile using `flex file.l` and `gcc lex.yy.c -o output -lfl`.

• Check the counting definition used by the program, especially for spaces, tabs, and final lines.